# Linux Ext4 Internals: Architecture, Features, and Performance Tuning

## Day 5: Debugging, Maintenance & Recovery

**Duration**

3.5 hours

**Level**

Expert

**Instructor**

Chandrashekar Babu

Email: **training@chandrashekar.info**

Website: **https://www.chandrashekar.info/** | **https://www.slashprog.com/**

# What We'll Cover Today

## Module 9: Debugging and Recovery Tools (1.5 hours)

- debugfs and e2fsck internals
- Kernel oops/crash debugging
- Performance profiling (perf, systemtap, eBPF)
- Code-level debugging (tracepoints, dynamic debug)
- Recovery from power loss and corruption

## Module 10: Maintenance and Future Development (1 hour)

- Online resize and maintenance
- Backup and restore strategies
- Development workflow
- Comparison with XFS/Btrfs

## Module 11: Summary & Expert Discussion (30 min)

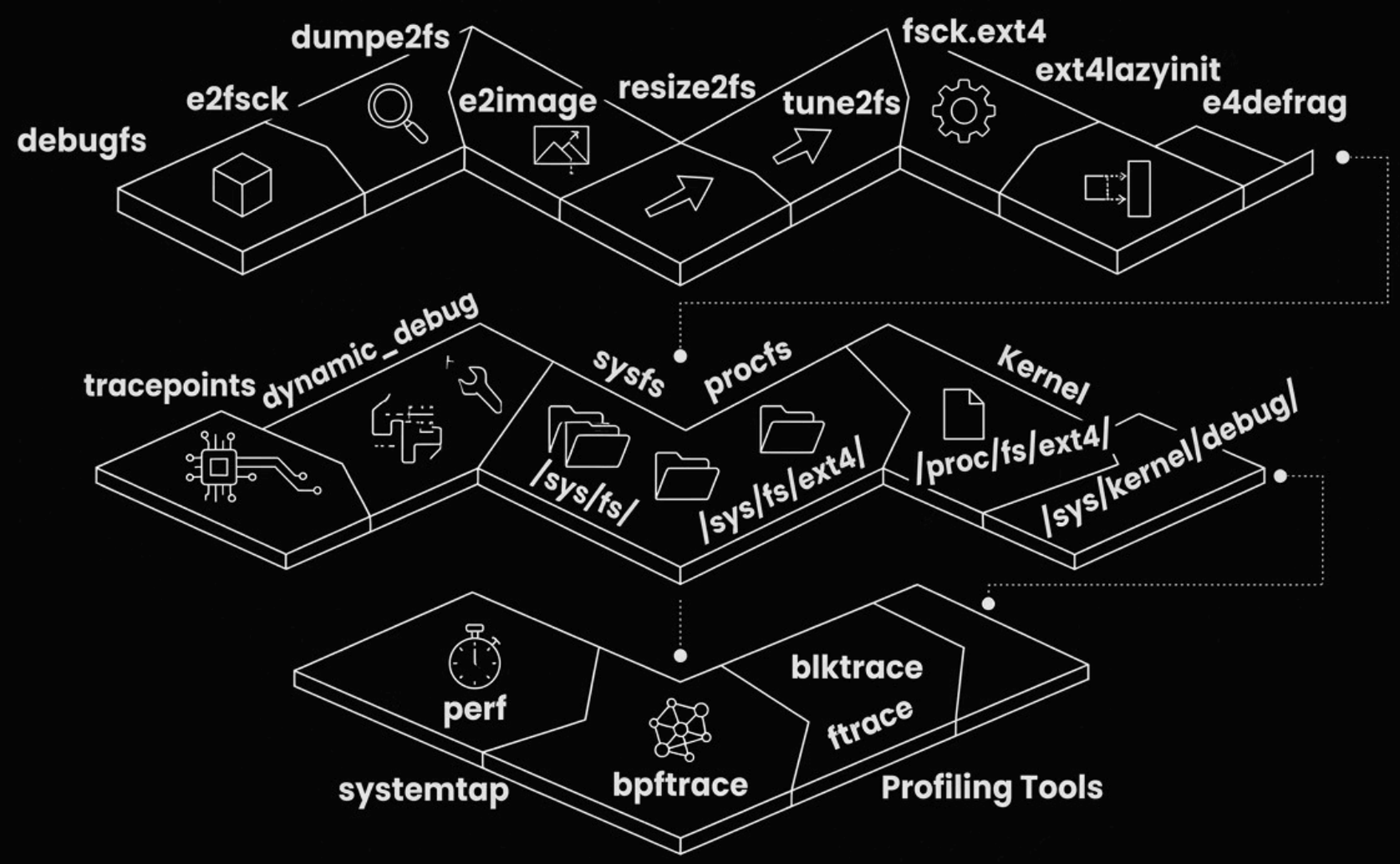
- Deployment best practices
- Performance checklist
- Community resources
- Final Q&A

## Hands-On Labs (30 minutes)

- Corruption recovery simulation
- Journal replay analysis
- Patch submission workflow

**Break:** 10 minutes between modules

# Ext4 Debugging Ecosystem




The Ext4 debugging ecosystem spans three layers: user-space tools for direct filesystem inspection, kernel interfaces for low-level access, and profiling tools for performance analysis.

# debugfs Deep Dive

## Interactive Filesystem Debugger — Core Commands

```
# Open filesystem
debugfs /dev/sda1
debugfs -w /dev/sda1 # Read-write mode
debugfs -c /dev/sda1 # Use cache (faster)

# Essential commands
debugfs: stats # Superblock statistics
debugfs: show_super_stats -h # Detailed superblock
debugfs: group_info # Block group info
debugfs: inode # Display inode
debugfs: cat # Print file content
debugfs: dump # Extract file
debugfs: logdump -a # Dump journal
debugfs: ls -l # List directory with details
debugfs: extents # Show extent tree
debugfs: blocks # Show block mapping
debugfs: stat # File status
debugfs: ncheck # Path from inode
debugfs: icheck # Inode from block
debugfs: find # Search for file
debugfs: htree # Show HTree structure
```

# debugfs Advanced Usage

```
# Scripting with debugfs
debugfs -R "stats" /dev/sda1
debugfs -R "dump /etc/passwd /tmp/passwd" /dev/sda1

# Batch operations
cat > debug_cmds.txt << EOF
cd /home
ls -l
stat important_file
blocks important_file
quit
EOF
debugfs -f debug_cmds.txt /dev/sda1

# Find deleted files
debugfs -R "ls -d /path" /dev/sda1 # List deleted entries
debugfs -R "undel " /dev/sda1 # Undelete if possible
```

# debugfs Internals

## How debugfs Works — Architecture

```
// debugfs uses libext2fs library
struct ext2_filsys {
    io_channel        io;         /* I/O channel */
    struct ext2_super_block *super;   /* Superblock */
    struct ext2_group_desc *group_desc; /* Group descriptors */
    struct ext2_inode   *inode_table; /* Inode table */
    blk64_t           blocks_count; /* Total blocks */
    blk64_t           inode_count;  /* Total inodes */
};

// Reading an inode
errcode_t ext2fs_read_inode(ext2_filsys fs, ext2_ino_t ino,
                            struct ext2_inode *inode) {
    // Locate inode in table
    group = (ino - 1) / fs->super->s_inodes_per_group;
    offset = ((ino - 1) % fs->super->s_inodes_per_group)
            * EXT2_INODE_SIZE(fs->super);
    // Read from disk
    ret = io_channel_read_blk(fs->io,
        fs->group_desc[group].bg_inode_table,
        offset, 1, inode);
    // Verify checksum if present
    if (EXT2_HAS_RO_COMPAT_FEATURE(fs->super,
        EXT4_FEATURE_RO_COMPAT_METADATA_CSUM))
        ext2fs_verify_inode_csum(fs, ino, inode);
    return 0;
}
```

```
// Extent tree traversal
static void dump_extents(ext2_filsys fs, struct ext2_inode *inode) {
 struct ext4_extent_header *eh;
 struct ext4_extent *ext;
 eh = (struct ext4_extent_header *)&inode->i_block[0];
 if (eh->eh_magic != EXT4_EXT_MAGIC) {
 printf("Not an extent file\n");
 return;
 }
 // Recursively traverse
 dump_extent_node(fs, eh, 0, 0);
}
```

# e2fsck Internals

## Filesystem Checker Architecture — Check Phases

```
// e2fsck main loop
int main(int argc, char **argv) {
    // Phase 1: Check blocks, inodes, and sizes
    e2fsck_pass1(ctx);
    // Phase 2: Check directory structure
    e2fsck_pass2(ctx);
    // Phase 3: Check directory connectivity
    e2fsck_pass3(ctx);
    // Phase 4: Check reference counts
    e2fsck_pass4(ctx);
    // Phase 5: Check group summary info
    e2fsck_pass5(ctx);
    // Phase 6: Salvage lost+found
    e2fsck_pass6(ctx);
    return 0;
}
```

```
// Pass 1: Inode and Block Validation
static void e2fsck_pass1(e2fsck_t ctx) {
 for (ino = 1; ino <= fs->super->s_inodes_count; ino++) {
 ext2fs_read_inode(fs, ino, &inode);
 if (inode.i_links_count == 0)
 handle_unlinked_inode(ctx, ino, &inode);
 if (ext2fs_inode_has_valid_blocks(&inode))
 check_blocks(ctx, ino, &inode);
 if (inode.i_file_acl)
 check_ext_attr(ctx, ino, &inode);
 if (inode.i_flags & EXT4_EXTENTS_FL)
 check_extent_tree(ctx, ino, &inode);
 }
}
```

# e2fsck Repair Strategies

| Problem Detection | Symptom | Repair |
|---|---|---|
| Bad superblock | Magic mismatch | Use backup |
| Invalid inode | Checksum fail | Clear inode |
| Block bitmap errors | Free/used mismatch | Rebuild bitmap |
| Directory loop | Depth > max | Break loop |
| Orphan inodes | Link count 0 | Clear or link |

# e2fsck Recovery Modes

## Interactive and Automated Recovery

### Options and Modes

```
# Automatic repair (safe)
e2fsck -p /dev/sda1  # Preen mode - fix safe errors
e2fsck -y /dev/sda1  # Yes to all questions
e2fsck -n /dev/sda1  # No to all (read-only check)

# Force check regardless of mount count
e2fsck -f /dev/sda1

# Verbose output
e2fsck -v /dev/sda1

# Background check (online)
e2fsck -c /dev/sda1  # Check for bad blocks

# Debug mode
e2fsck -d /dev/sda1
```

### Recovery Scenarios

```
# Superblock recovery
e2fsck -b 32768 /dev/sda1 # Use backup superblock

# Block group descriptor recovery
e2fsck -B 4096 /dev/sda1 # Specify block size

# Resize after corruption
e2fsck -f /dev/sda1
resize2fs /dev/sda1

# Save context for debugging
e2image -r /dev/sda1 /tmp/ext4-image
e2fsck -F /tmp/ext4-image # Check the image
```

# Kernel Oops/Crash Debugging

## Analysing Kernel Panics — Oops Message Structure

```
[ 123.456789] BUG: unable to handle kernel NULL pointer dereference at 0000000000000000
[ 123.456790] IP: ext4_evict_inode+0x123/0x456 [ext4]
[ 123.456791] PGD 0 P4D 0
[ 123.456792] Oops: 0000 [#1] SMP PTI
[ 123.456793] CPU: 2 PID: 12345 Comm: rm Tainted: G W OE 5.15.0
[ 123.456794] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996)
[ 123.456795] RIP: 0010:ext4_evict_inode+0x123/0x456 [ext4]
[ 123.456796] RSP: 0018:ffffc9000123fdf8 EFLAGS: 00010246
[ 123.456797] RAX: 0000000000000000 RBX: ffff888123456789 RCX: 0000000000000000
[ 123.456798] RDX: 0000000000000000 RSI: 0000000000000000 RDI: ffff888123456789
```

# Kernel Oops — Debugging Steps

```
# 1. Decode the oops
scripts/decodecode < oops.txt

# 2. Find function offset
gdb vmlinux
(gdb) list *(ext4_evict_inode+0x123)

# 3. Examine register values
# RAX=0 indicates NULL pointer dereference
# RDI points to inode structure

# 4. Check kernel logs
dmesg | grep -A20 "Oops"
journalctl -k -f

# 5. Use crash utility
crash vmlinux vmcore
crash> bt
crash> log
crash> dis -l ext4_evict_inode
```

# Kernel Crash Dump Analysis

## Setting Up kdump

### Configuration

```
# Install kdump tools
apt-get install kdump-tools   # Debian/Ubuntu
yum install kexec-tools       # RHEL/CentOS

# Configure crash kernel
cat >> /etc/default/grub << EOF
GRUB_CMDLINE_LINUX_DEFAULT="crashkernel=256M"
EOF
update-grub

# Configure kdump
cat > /etc/kdump.conf << EOF
path /var/crash
core_collector makedumpfile -c -d 31
default reboot
EOF
systemctl enable kdump
systemctl start kdump
```

### Analysing vmcore

```
# Start crash analysis
crash /usr/lib/debug/lib/modules/$(uname -r)/vmlinux \
 /var/crash/vmcore

# Useful crash commands
crash> bt # Backtrace of crashed task
crash> bt -a # All CPUs backtrace
crash> log # Kernel log buffer
crash> ps # Process list
crash> files # Open files
crash> kmem -i # Memory info
crash> struct ext4_inode_info ffff888123456789
crash> dis -l ext4_evict_inode
crash> search -t ext4_inode_info
```

# Performance Profiling with perf

## Ext4 Performance Analysis — perf Event Categories

```
# List ext4 events
perf list ext4:
# ext4:ext4_es_lookup_extent_enter
# ext4:ext4_es_lookup_extent_exit
# ext4:ext4_ext_map_blocks_enter
# ext4:ext4_ext_map_blocks_exit
# ext4:ext4_da_write_begin
# ext4:ext4_da_write_end
# ext4:ext4_writepages
# ext4:ext4_sync_file_enter
# ext4:ext4_sync_file_exit

# JBD2 events
perf list jbd2:
# jbd2:jbd2_start_commit
# jbd2:jbd2_commit_locking
# jbd2:jbd2_commit_flushing
# jbd2:jbd2_end_commit
```

```
# Trace extent lookups
perf record -e ext4:ext4_es_lookup_extent_* -a -- sleep 10
perf report

# Count journal commits
perf stat -e jbd2:jbd2_commit -a -- sleep 60

# Flame graph of ext4 operations
perf record -e ext4:* -a -g -- sleep 30
perf script | stackcollapse-perf.pl | flamegraph.pl > ext4.svg

# CPU sampling with ext4 focus
perf record -F 99 -a -g -- filter 'comm == "kworker" || comm == "jbd2"'
perf report --sort comm,dso -g graph
```

# eBPF and bpftrace for Ext4

## Dynamic Tracing Examples — Trace File Opens

```
# Trace all ext4 file opens
cat > ext4_open.bt << 'EOF'
kprobe:ext4_file_open {
    printf("open: %s (pid=%d)\n",
        str(arg1->f_path.dentry->d_name.name), pid);
}
kretprobe:ext4_file_open {
    printf("open returned: %d\n", retval);
}
EOF
bpftrace ext4_open.bt
```

```
# Track ext4 write latency
cat > ext4_write_latency.bt << 'EOF'
kprobe:ext4_file_write_iter {
 @start[tid] = nsecs;
}
kretprobe:ext4_file_write_iter /@start[tid]/ {
 $lat = nsecs - @start[tid];
 @latency_us = hist($lat / 1000);
 @size[tid] = arg2->count;
 delete(@start[tid]);
}
interval:s:10 {
 print(@latency_us);
 clear(@latency_us);
}
EOF
bpftrace ext4_write_latency.bt
```

# eBPF — Extent Cache Miss Analysis

```
# Track extent cache misses
cat > extent_miss.bt << 'EOF'
kprobe:ext4_ext_find_extent {
 @calls[tid] = count();
}
kprobe:ext4_ext_map_blocks /arg3 == 0/ {
 @misses[tid] = count();
 printf("Extent cache miss: block=%llu\n", arg2);
}
EOF
bpftrace extent_miss.bt
```

# Tracepoints and Dynamic Debug

## Kernel Tracepoints — Enabling Ext4 Tracepoints

### Enabling Tracepoints

```
# List available tracepoints
ls /sys/kernel/debug/tracing/events/ext4/

# Enable specific tracepoints
echo 1 >
/sys/kernel/debug/tracing/events/ext4/ext4_da_write_begin/ena
ble
echo 1 >
/sys/kernel/debug/tracing/events/ext4/ext4_da_write_end/enabl
e

# Enable all ext4 tracepoints
echo 1 > /sys/kernel/debug/tracing/events/ext4/enable

# Read trace output
cat /sys/kernel/debug/tracing/trace_pipe

# Filter by specific inode
echo "ino == 12345" >
/sys/kernel/debug/tracing/events/ext4/filter
```

### Dynamic Debug

```
# Enable ext4 debug messages
echo "file fs/ext4/* +p" >
/sys/kernel/debug/dynamic_debug/control

# Enable specific functions
echo "func ext4_map_blocks +p" >
/sys/kernel/debug/dynamic_debug/control

# Disable debugging
echo "file fs/ext4/* -p" >
/sys/kernel/debug/dynamic_debug/control

# Check current settings
cat /sys/kernel/debug/dynamic_debug/control | grep ext4
```

### Custom Debug Prints

```
// Kernel debug macros
#define ext4_debug(fmt, ...) \
 do { \
 if (ext4_debug_flag) \
 printk(KERN_DEBUG "EXT4-fs: " fmt, ##__VA_ARGS__); \
 } while (0)

// Enable with mount option
mount -o debug /dev/sda1 /mnt
```

# Essential Ext4 Tools

## Tool Ecosystem Overview

### dumpe2fs

```
# Dump filesystem information
dumpe2fs /dev/sda1
dumpe2fs -h /dev/sda1  # Superblock only
dumpe2fs -b /dev/sda1  # Show bad blocks
dumpe2fs -o superblock=32768 /dev/sda1  # Use backup superblock

# Scriptable output
dumpe2fs -h /dev/sda1 | grep -E "Block size|Inode size"
```

### e2image

```
# Create metadata image (for debugging)
e2image -r /dev/sda1 /tmp/ext4-image      # Raw image
e2image -Q /dev/sda1 /tmp/ext4-image.qcow2 # QCOW2 format

# Restore from image
e2image -r /tmp/ext4-image /dev/sdb1

# Analyse image
debugfs /tmp/ext4-image
e2fsck /tmp/ext4-image
```

### ext4lazyinit

```
# Check lazy initialization status
dumpe2fs /dev/sda1 | grep lazy

# Force completion
tune2fs -E lazy_itable_init=0 /dev/sda1

# Monitor progress
cat /proc/fs/ext4/sda1/lazy_itable_init_status
```

# Recovery from Power Loss

## Journal Replay Deep Dive — Recovery Code Path

```
// Journal recovery entry point
int jbd2_journal_recover(journal_t *journal) {
 journal_superblock_t *sb = journal->j_superblock;
 tid_t first_tid, last_tid;
 int err;

 // Get recovery range
 first_tid = sb->s_first;
 last_tid = sb->s_last;

 // Scan transactions
 for (tid = first_tid; tid <= last_tid; tid++) {
 // Read transaction
 transaction = jbd2_journal_read_transaction(journal, tid);

 // Verify transaction integrity
 if (!jbd2_transaction_valid(transaction)) {
 jbd2_journal_clear_revoke(journal);
 continue;
 }

 // Replay blocks
 for (i = 0; i < transaction->t_nr_buffers; i++) {
 bh = transaction->t_buffers[i];
 // Write to main filesystem
 err = sync_dirty_buffer(bh);
 if (err) return err;
 }

 // Update journal head
 journal->j_head = tid + 1;
 }
 return 0;
}
```

# Error Injection Testing

```
// Fault injection for testing recovery
static int ext4_inject_error(struct super_block *sb, int error) {
 struct ext4_sb_info *sbi = EXT4_SB(sb);

 if (!test_opt(sb, ERRORS_CONT))
 return 0;

 // Simulate crash during journal commit
 if (sbi->s_inject_error == EXT4_INJ_JOURNAL_COMMIT) {
 printk(KERN_ERR "EXT4: Injecting journal commit failure\n");
 return -EIO;
 }

 // Simulate power loss
 if (sbi->s_inject_error == EXT4_INJ_POWER_LOSS) {
 printk(KERN_ERR "EXT4: Injecting simulated power loss\n");
 while (1) cpu_relax(); // Hang the kernel
 }

 return 0;
}
```

# Data Corruption Recovery

## Handling Memory Errors (DDR Faults) — Integrity Checks

```
// Detect corruption during read
static int ext4_verify_read(struct inode *inode, struct page *page,
                            unsigned int off, unsigned int len) {
    struct ext4_inode_info *ei = EXT4_I(inode);
    sector_t block;

    // Check if data is encrypted
    if (IS_ENCRYPTED(inode)) {
        // Verify decryption
        if (fscrypt_verify_contents(inode, page))
            return -EIO;
    }

    // For metadata, verify checksums
    if (S_ISREG(inode->i_mode) == 0) {
        block = page->index << (PAGE_SHIFT - inode->i_blkbits);
        if (ext4_verify_metadata_block(inode->i_sb, block, page))
            return -EIO;
    }

    return 0;
}
```

```
# Check for memory errors
dmesg | grep -i "mce\|hardware error\|EDAC"

# Check for filesystem corruption
dumpe2fs /dev/sda1 | grep -i "error\|corrupt"

# Use e2fsck with badblocks
badblocks -v /dev/sda1 > badblocks.txt
e2fsck -l badblocks.txt /dev/sda1

# Recover from backup superblock
e2fsck -b 32768 /dev/sda1

# Extract data from corrupted files
debugfs -R "cat /path/to/file" /dev/sda1 > recovered_file
```

# Hands-On Lab 1 — Corruption Recovery

## Setup Test Environment

```
# Create test filesystem
dd if=/dev/zero of=/tmp/corrupt.img bs=1M count=1024
mkfs.ext4 -F /tmp/corrupt.img

# Mount and populate
mkdir /mnt/corrupt
mount -o loop /tmp/corrupt.img /mnt/corrupt
cd /mnt/corrupt

# Create test data
for i in {1..1000}; do
 echo "Data for file $i" > file$i
done
mkdir -p deep/dir/structure
echo "important data" > important.txt
sync

# Get superblock location
dumpe2fs /tmp/corrupt.img | grep "Superblock backups"
umount /mnt/corrupt
```

# Lab 1 — Corrupt Superblock & Inode Table

## Corrupt Superblock

```
# Corrupt superblock (block 0)
dd if=/dev/zero of=/tmp/corrupt.img bs=1k count=1
conv=notrunc

# Try to mount (should fail)
mount -o loop /tmp/corrupt.img /mnt/corrupt
# Expected: mount: /mnt/corrupt: wrong fs type,
#           bad option, bad superblock

# Recover using backup superblock
mount -o loop,sb=32768 /tmp/corrupt.img /mnt/corrupt

# Fix permanently
umount /mnt/corrupt
e2fsck -b 32768 /tmp/corrupt.img
mount -o loop /tmp/corrupt.img /mnt/corrupt
```

## Corrupt Inode Table

```
# Find inode table location
dumpe2fs /tmp/corrupt.img | grep "Inode table"

# Corrupt inode table (example: group 0)
dd if=/dev/urandom of=/tmp/corrupt.img \
 bs=4k seek=100 count=10 conv=notrunc

# Check and recover
e2fsck -f /tmp/corrupt.img
# Follow prompts to repair
```

# Module 9 Summary

## Debugging and Recovery Key Takeaways

### Debug Tools

- **debugfs**: Interactive filesystem inspection
- **e2fsck**: Filesystem repair (5-phase algorithm)
- **perf**: Performance profiling with tracepoints
- **bpftrace**: Dynamic tracing for custom analysis

### Recovery Techniques

- **Journal replay**: Automatic on mount
- **Superblock backup**: Multiple copies for recovery
- **e2image**: Save metadata for offline analysis
- **Error injection**: Test recovery paths

### Debugging Methods

- **Oops analysis**: Decode kernel panic messages
- **kdump**: Capture vmcore for post-mortem
- **Dynamic debug**: Enable on-demand logging
- **Tracepoints**: Low-overhead instrumentation

### Best Practices

- Regular e2fsck checks
- Backup critical metadata with e2image
- Monitor for hardware errors (ECC, bad blocks)
- Test recovery procedures regularly

# Module 10 — Online Resize

## Growing Filesystems Online — Resize Operations

```
# Check current size
df -h /mnt/data
dumpe2fs /dev/sda1 | grep "Block count"

# Extend partition first
fdisk /dev/sda1      # Increase partition size
partprobe /dev/sda1

# Online resize
resize2fs /dev/sda1
resize2fs /dev/sda1 10G  # Resize to 10GB

# Check new size
df -h /mnt/data
```

```
// Online resize entry point
int ext4_group_extend(struct super_block *sb,
                      struct ext4_super_block *es,
                      ext4_fsblk_t n_blocks_count) {
    struct ext4_sb_info *sbi = EXT4_SB(sb);
    ext4_fsblk_t o_blocks_count;
    o_blocks_count = ext4_blocks_count(es);

    // Validate new size
    if (n_blocks_count <= o_blocks_count)
        return -EINVAL;

    // Add new block groups
    while (o_blocks_count < n_blocks_count) {
        err = ext4_add_new_group_desc(sb, es, &o_blocks_count);
        if (err) return err;
    }

    // Update superblock
    ext4_blocks_count_set(es, n_blocks_count);
    ext4_mark_super_dirty(sb);
    return 0;
}
```

```
# Cannot shrink online (must unmount)
resize2fs -M /dev/sda1 # Shrink to minimum (offline)

# Check for resize flags
tune2fs -l /dev/sda1 | grep "Resize"
```

# Backup and Restore Strategies

## Ext4 Backup Approaches

### File-Level Backup

```
# rsync with ext4 attributes
rsync -avAX --delete /source/ /backup/
# -A: preserve ACLs
# -X: preserve extended attributes

# tar with xattr support
tar --xattrs -cvpf backup.tar /source/

# dump/restore (ext2/3/4 native)
dump -0uf /backup/dump.dump /dev/sda1
restore -rf /backup/dump.dump
```

### Filesystem-Level Backup

```
# Create filesystem image
dd if=/dev/sda1 of=/backup/ext4.img bs=4M status=progress

# Compressed image
dd if=/dev/sda1 bs=4M | gzip -c > /backup/ext4.img.gz

# Sparse image for thin provisioning
cp --sparse=always /dev/sda1 /backup/ext4-sparse.img

# Incremental backup
e2image -r -Q /dev/sda1 /backup/ext4.qcow2
```

### Metadata Backup

```
# Backup superblock and group descriptors
e2image -r /dev/sda1 /backup/ext4-metadata.img

# Restore metadata
e2image -r /backup/ext4-metadata.img /dev/sdb1

# Create rescue media
e2image -Q /dev/sda1 /backup/ext4-rescue.qcow2
```

# Development Workflow

## Contributing to Ext4 — Getting Started

### Repository Setup

```
# Clone kernel repository
git clone
git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux

# Set up development branch
git remote add stable
git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
git checkout -b ext4-dev

# Build kernel
make defconfig
make -j$(nproc)
```

### Code Structure

```
fs/ext4/
├── balloc.c    # Block allocation
├── dir.c       # Directory operations
├── extents.c   # Extent tree
├── file.c      # File operations
├── ialloc.c    # Inode allocation
├── inode.c     # Inode operations
├── ioctl.c     # IOCTL handlers
├── mballoc.c   # Multiblock allocator
├── super.c     # Superblock operations
├── xattr.c     # Extended attributes
└── crypto.c    # Encryption
```

```
# Prepare patch
git add fs/ext4/my_feature.c
git commit -s -v # -s: Signed-off-by

# Generate patch
git format-patch -1 -o patches/

# Check with scripts
scripts/checkpatch.pl patches/0001-*.patch

# Send to mailing list
git send-email --to=linux-ext4@vger.kernel.org patches/
```

# Patch Submission Workflow

## Contribution Guidelines — Patch Requirements

```
// Example patch with proper formatting
From: Developer Name
Date: Mon, 1 Jan 2024 12:00:00 -0800
Subject: [PATCH] ext4: Add new feature

Add description of what the patch does and why.
Explain the technical details and any potential side effects.

Signed-off-by: Developer Name
---
 fs/ext4/super.c | 10 +++++++++-
 1 file changed, 9 insertions(+), 1 deletion(-)

diff --git a/fs/ext4/super.c b/fs/ext4/super.c
index abc123..def456 100644
--- a/fs/ext4/super.c
+++ b/fs/ext4/super.c
@@ -1234,6 +1234,7 @@ static int ext4_fill_super(struct super_block *sb,
 {
    struct ext4_sb_info *sbi;
    int ret;
+   int new_feature;

    sbi = kzalloc(sizeof(*sbi), GFP_KERNEL);
    if (!sbi)
@@ -1245,6 +1246,13 @@ static int ext4_fill_super(struct super_block *sb,
    sb->s_fs_info = sbi;
    sbi->s_sb = sb;

+   /* Enable new feature */
+   if (test_opt(sb, NEW_FEATURE)) {
+       new_feature = 1;
+       ext4_set_feature_new_feature(sb);
+       ext4_msg(sb, KERN_INFO, "New feature enabled");
+   }
+
    ret = ext4_load_journal(sb, es, journal_devnum);
    if (ret)
        goto out_fail;
```

```
# Check for common issues
scripts/checkpatch.pl --strict patches/*.patch

# Run kernel tests
make -C tools/testing/selftests TARGETS=ext4

# Build with all warnings
make W=1 fs/ext4/

# Test with CONFIG_EXT4_DEBUG
make fs/ext4/ CONFIG_EXT4_DEBUG=y
```

# Checkpointing and Crash Simulation Lab

## Simulating Filesystem Crashes — Setup

```
# Create test filesystem with journal
dd if=/dev/zero of=/tmp/crash.img bs=1M count=1024
mkfs.ext4 -F -J size=64 /tmp/crash.img

# Mount with debug options
mkdir /mnt/crash
mount -o loop,debug /tmp/crash.img /mnt/crash

# Enable detailed journal tracing
echo 1 > /sys/kernel/debug/tracing/events/jbd2/enable
echo 1 > /sys/kernel/debug/tracing/events/ext4/enable
```

# Crash Simulation Script

```bash
#!/bin/bash
# simulate_crash.sh

# Create workload in background
(
    cd /mnt/crash
    for i in {1..1000}; do
        echo "Data $i" > file$i
        sync
        sleep 0.1
    done
) &

# Wait for some writes
sleep 2

# Simulate power loss (force unmount without journal replay)
umount -l /mnt/crash

# Check journal state
debugfs -R "logdump -a" /tmp/crash.img > journal_dump.txt

# Replay journal automatically on next mount
mount -o loop /tmp/crash.img /mnt/crash
dmesg | grep -i "replay\|recovery"

# Check consistency
umount /mnt/crash
e2fsck -f /tmp/crash.img
```

```bash
#!/bin/bash
# analyze_journal.sh

# Extract transaction information
debugfs -R "logdump -a" /tmp/crash.img | \
 grep -E "Transaction|COMMIT|DESC" > trans.txt

# Count committed vs incomplete transactions
echo "=== Transaction Analysis ==="
grep "Transaction ID" trans.txt | sort | uniq -c
grep "COMMIT" trans.txt | wc -l
grep "DESC" trans.txt | wc -l

# Check for orphan inodes
debugfs -R "ls -d /" /tmp/crash.img
```

# Comparison with XFS/Btrfs

## Filesystem Comparison Matrix

| Feature | Ext4 | XFS | Btrfs |
|---|---|---|---|
| **Journaling** | Metadata + optional data | Metadata only | Copy-on-write |
| **Max Filesystem** | 1 EB | 8 EB | 16 EB |
| **Max File Size** | 16 TB | 8 EB | 16 EB |
| **Snapshots** | No | No | Yes |
| **Checksums** | Metadata only | Metadata only | Data + metadata |
| **Compression** | No | No | Yes (zlib, lzo, zstd) |
| **Deduplication** | No | No | Yes |
| **RAID Support** | No | No | Built-in |
| **Online Defrag** | Yes | Yes | Yes |
| **Subvolumes** | No | No | Yes |
| **Casefold** | Yes | No | No |
| **Encryption** | fscrypt | fscrypt (partial) | fscrypt |
| **Stability** | Very high | High | Mature |

**Recovery Approach:** Ext4 uses journal-based recovery (replay committed transactions, discard incomplete). XFS uses log-based recovery (similar structure). Btrfs uses copy-on-write + tree recovery (find latest valid root, rollback).

# Future Features

## Ext4 Roadmap — In Development

```
// Compression support (planned)
struct ext4_compression_info {
    __u32 ci_algorithm;  /* lzo, zstd, etc */
    __u32 ci_level;      /* Compression level */
    __u64 ci_orig_size;  /* Original size */
};

// Subvolume support (research phase)
struct ext4_subvolume {
    __u64 sv_id;        /* Subvolume ID */
    __u64 sv_parent;    /* Parent subvolume */
    char  sv_name[256]; /* Subvolume name */
};

// Zone-aware allocation (NVMe ZNS)
struct ext4_zone_info {
    __u32 zi_zone_type;      /* Conventional/Sequential */
    __u64 zi_zone_size;      /* Zone size in blocks */
    __u64 zi_write_pointer;  /* Current write position */
};
```

```
# Enable experimental features
mkfs.ext4 -O test_fs /dev/sda1

# Current experimental flags
# - fast_commit: Faster journal commits
# - orphan_file: Improved orphan handling
# - ea_inode: Extended attribute inodes
```

# Module 10 Summary

## Maintenance and Development Key Takeaways

### Online Operations

- **Online resize**: Grow filesystem without unmount
- **Online defrag**: e4defrag for file-level defrag
- **Online tune**: tune2fs for live parameter changes

### Backup Strategies

- **File-level**: rsync, tar with xattr support
- **Image-level**: dd, e2image for metadata
- **Incremental**: Combine with LVM snapshots

### Development Workflow

- **Clone**: Linux kernel git repository
- **Code**: fs/ext4/ structure
- **Test**: kernel selftests, custom workloads
- **Submit**: **linux-ext4@vger.kernel.org**

### Filesystem Comparison

- **Ext4**: General purpose, stable, simple
- **XFS**: Large files, high performance
- **Btrfs**: Advanced features, snapshots, checksums

### Future Directions

- **Compression**: Save space at CPU cost
- **Subvolumes**: Filesystem-level isolation
- **Zone awareness**: Optimised for ZNS SSDs

# Module 11 — Deployment Best Practices

## Production Configuration Checklist — Filesystem Creation

```
# Optimal mkfs options
mkfs.ext4 -O extent,flex_bg,metadata_csum,64bit \
        -E stride=64,stripe_width=128 \
        -J size=512 \
        -m 1 \
        /dev/sda1
```

## Mount Options by Workload

| Workload | Mount Options |
|---|---|
| **Database** | noatime,nodiratime,data=ordered,barrier=1,noauto_da_alloc |
| **Web Server** | noatime,data=ordered,barrier=1 |
| **Backup** | noatime,data=writeback,nobarrier |
| **VM Storage** | noatime,data=ordered,barrier=1 |
| **Container** | noatime,data=ordered,prjquota |
| **Desktop** | defaults,noatime |
| **Embedded** | noatime,data=ordered,nobarrier,noauto_da_alloc |

```
# /etc/sysctl.d/99-ext4.conf
vm.dirty_ratio = 20
vm.dirty_background_ratio = 10
vm.dirty_expire_centisecs = 3000
vm.vfs_cache_pressure = 100

# I/O scheduler (by device)
echo "deadline" > /sys/block/sda/queue/scheduler # HDD
echo "none" > /sys/block/nvme0n1/queue/scheduler # NVMe
```

# Performance Checklist — Database

## MySQL/PostgreSQL

```
# Filesystem
mkfs.ext4 -O extent,flex_bg,metadata_csum -J size=512 /dev/sda1

# Mount
mount -o noatime,data=ordered,barrier=1,noauto_da_alloc /dev/sda1 /var/lib/mysql

# Kernel
echo 0 > /proc/sys/vm/dirty_ratio
echo 10 > /proc/sys/vm/dirty_background_ratio

# Database
# innodb_flush_method=O_DIRECT
# innodb_flush_log_at_trx_commit=1
```

# Performance Checklist — Web Server & Backup

## Web Server (Nginx/Apache)

```
# Filesystem
mkfs.ext4 -O extent,flex_bg -J size=256 /dev/sda1

# Mount
mount -o noatime,data=ordered /dev/sda1 /var/www

# Kernel
echo 20 > /proc/sys/vm/dirty_ratio
echo 15 > /proc/sys/vm/dirty_background_ratio
```

## Backup Server

```
# Filesystem
mkfs.ext4 -O extent,flex_bg -J size=2048 /dev/sda1

# Mount
mount -o noatime,data=writeback,nobarrier /dev/sda1 /backup

# Kernel
echo 50 > /proc/sys/vm/dirty_ratio
echo 30 > /proc/sys/vm/dirty_background_ratio
```

# Monitoring and Alerting

## Key Metrics to Monitor — Filesystem Metrics

```
# Usage
df -h /mnt/data
dumpe2fs -h /dev/sda1 | grep -E "Free blocks|Free inodes"

# Fragmentation
filefrag /mnt/data/*
e4defrag -c /mnt/data

# Error counts
cat /sys/fs/ext4/sda1/errors_count
dmesg | grep -i "ext4 error"

# Journal status
cat /proc/fs/jbd2/sda1-8/info
```

```
# node_exporter ext4 metrics
- job_name: 'ext4'
  static_configs:
  - targets: ['localhost:9100']
  metrics_path: /metrics
  relabel_configs:
  - source_labels: [__name__]
    regex: 'node_filesystem_(.*)'
    action: keep
```

```
groups:
- name: ext4_alerts
 rules:
 - alert: Ext4FilesystemFull
 expr: node_filesystem_avail_bytes{fstype="ext4"} /
 node_filesystem_size_bytes < 0.1
 annotations:
 summary: "Filesystem {{ $labels.mountpoint }} is {{ $value }}% full"
 - alert: Ext4Errors
 expr: node_ext4_errors > 0
 annotations:
 summary: "Ext4 errors detected on {{ $labels.device }}"
```

# Community Resources

## Getting Help and Contributing

### Mailing Lists

- linux-ext4@vger.kernel.org — Development discussions
- linux-fsdevel@vger.kernel.org — Filesystem development
- kernelnewbies@kernelnewbies.org — New contributor help

### IRC Channels

- **#ext4** on irc.libera.chat — Real-time help
- **#kernel** on irc.libera.chat — Kernel development

### Bug Tracking

- Kernel Bugzilla (Component: ext4)
- GitHub Issues

### Documentation

- **Kernel docs**: kernel.org/doc/html/latest/filesystems/ext4.html
- **Ext4 Wiki**: ext4.wiki.kernel.org
- **e2fsprogs**: git.kernel.org/pub/scm/fs/ext2/e2fsprogs.git

### Tools and Utilities

```
# e2fsprogs (maintained by Ted Ts'o)
git clone git://git.kernel.org/pub/scm/fs/ext2/e2fsprogs.git

# fio benchmark
git clone https://github.com/axboe/fio.git

# blktrace
git clone
git://git.kernel.org/pub/scm/linux/kernel/git/axboe/blktrace.git
```

# Future Directions

## What's Coming in Ext4 — Fast Commits

```
// Reducing fsync latency
struct ext4_fast_commit {
    __le32 fc_magic;    /* Fast commit magic */
    __le32 fc_version;  /* Version number */
    __le32 fc_ino;      /* Inode number */
    __le32 fc_tag;      /* Tag (create, delete, etc) */
    __le32 fc_len;      /* Length of data */
    char   fc_data[0];  /* Variable-length data */
};
```

```
// Improved orphan inode handling
struct ext4_orphan_file {
    __le32 of_magic;    /* Orphan file magic */
    __le32 of_version;  /* Version */
    __le32 of_entries;  /* Number of entries */
    struct ext4_orphan_entry of_entry[]; /* Orphan entries */
};
```

## Future Enhancements

01

### Compression

Zstd, LZ4 support for data blocks

02

### Subvolumes

Lightweight container isolation

03

### DAX Improvements

Better persistent memory support

04

### ZNS Optimisation

Zone-aware allocation

05

### Rust Bindings

Memory-safe filesystem code

# Final Q&A Topics

## Expert Discussion Points

### Architecture Questions

- **Why not Btrfs?**: Ext4's stability vs feature trade-offs
- **Journaling evolution**: Ordered mode vs full data journaling
- **Extent tree depth**: When do we need 5 levels?
- **Delalloc trade-offs**: ENOSPC risk vs performance

### Performance Questions

- **NVMe optimisation**: What changes from SATA?
- **Memory pressure**: How to balance page cache vs application?
- **Journal sizing**: Formula for optimal journal size
- **Barrier disabling**: When is it really safe?

### Debugging Questions

- **Crash analysis**: Steps to debug production panic
- **Corruption causes**: Common failure modes
- **Recovery tools**: When to use each tool
- **Performance profiling**: Identifying bottlenecks

### Future Questions

- **Compression**: Will it be mainlined?
- **Subvolumes**: How will they compare to Btrfs?
- **Rust rewrite**: Is it possible?
- **Replacing ext4**: What would succeed it?

# Course Wrap-Up

## 5-Day Journey Summary

1

### Day 1: Fundamentals

On-disk structures, superblock, inodes. Mount internals, read-only handling.

2

### Day 2: Journaling & Metadata

Transaction lifecycle, recovery. Allocation algorithms, quotas.

3

### Day 3: I/O & Memory

Extent trees, delalloc, mballoc. Page cache, direct I/O, folios.

4

### Day 4: Advanced Features & Tuning

Encryption, bigalloc, casefold. Mount options, storage tuning.

5

### Day 5: Debugging & Maintenance

Tools, recovery, development. Best practices, future directions.

# Key Takeaways

## What You Should Remember

### Design Philosophy

- **Backward compatibility**: Mount Ext2/Ext3
- **Incremental improvement**: Stable, proven codebase
- **Feature flags**: Controlled feature adoption

### Critical Operations

- **Journal replay**: Automatic crash recovery
- **Delayed allocation**: Performance vs space trade-off
- **Extent tree**: Efficient large file handling

### Debugging Arsenal

- **debugfs**: First line inspection
- **e2fsck**: Filesystem repair
- **tracepoints**: Kernel-level insight
- **bpftrace**: Dynamic tracing

### Performance Optimisation

- **noatime**: Single biggest win
- **fstrim**: SSD maintenance
- **Journal tuning**: Workload-specific
- **Mount options**: Context-dependent

### Community

- **Mailing list**: Development discussions
- **Kernel docs**: Comprehensive reference
- **Source code**: Ultimate documentation

# Final Hands-On Challenge

## Putting It All Together — Scenario: Production Incident

You have a production server with: **2TB Ext4 filesystem on RAID10 (4x1TB HDD)**, MySQL database (200GB data), 24/7 operation, recent performance degradation, and occasional "Filesystem full" errors despite 20% free.

## Challenge Tasks

```
# 1. Diagnose the issue
# Check fragmentation, journal size, mount options

# 2. Plan recovery strategy
# Minimal downtime approach

# 3. Implement fixes
# Online operations only

# 4. Verify improvements
# Benchmark before/after

# 5. Document solution
# What would you change?
```

# Final Challenge — Expected Solution Areas

### Diagnose

- Check filefrag for fragmentation
- Evaluate journal size with dumpe2fs
- Review mount options (/proc/mounts)

### Tune

- Consider noatime, data=ordered
- Check discard vs fstrim
- Analyse dmesg for errors

### Optimise

- Consider online defrag with e4defrag
- Evaluate tune2fs parameter changes

# Certification and Next Steps

## Advanced Learning Paths

### Recommended Next Courses

01

**Linux Kernel Development** — Understanding kernel internals

02

**Storage Systems** — Block layer, NVMe, SCSI

03

**Performance Engineering** — Profiling, optimisation

### Community Involvement

- Join LKML (linux-kernel mailing list)
- Attend Linux Plumbers Conference (Filesystem track)
- Participate in kernel newbies mentoring
- Test -next kernel for ext4 changes

### Self-Study Resources

```
# Read the source
git clone
git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux/fs/ext4

# Study key files
ls -la | grep -E "(extents|inode|super|mballoc)"

# Follow commits
git log --oneline --follow extents.c

# Contribute
# Start with documentation fixes
# Move to bug fixes
# Then new features
```

# Thank You & Contact

## Questions & Feedback

### Resources

- **Slides**: https://github.com/chandrashekar-babu/linux-ext4-internals
- **Contact**: training@chandrashekar.info
- **LinkedIn:** https://www.linkedin.com/in/chandrashekarbabu/
- **YouTube:** https://www.youtube.com/@chandrashekarbabu
- **Website:** https://www.chandrashekar.info/

**End of Day 5 & Course** — Thank you for participating!

# Appendix — Quick Reference

## Ext4 Commands: Filesystem Creation & Information

```
# Filesystem Creation
mkfs.ext4 /dev/sda1 # Default
mkfs.ext4 -O extent,flex_bg,metadata_csum /dev/sda1 # Advanced
mkfs.ext4 -J size=512 -m 1 /dev/sda1 # Journal + reserved

# Filesystem Information
dumpe2fs /dev/sda1 # Full info
dumpe2fs -h /dev/sda1 # Superblock only
tune2fs -l /dev/sda1 # List parameters

# Filesystem Check
e2fsck -p /dev/sda1 # Automatic repair
e2fsck -f /dev/sda1 # Force check
e2fsck -b 32768 /dev/sda1 # Use backup superblock
```

# Appendix — Quick Reference (continued)

## Mount Options, Tuning & Online Operations

```
# Mount Options
mount -o defaults /dev/sda1 /mnt # Default
mount -o noatime,data=ordered /dev/sda1 /mnt # Balanced
mount -o noatime,data=writeback,nobarrier /dev/sda1 /mnt # Performance

# Tuning
tune2fs -c 30 /dev/sda1 # Max mount count
tune2fs -i 30d /dev/sda1 # Interval between checks
tune2fs -m 2 /dev/sda1 # Reserved blocks %
tune2fs -O ^metadata_csum /dev/sda1 # Disable feature

# Online Operations
resize2fs /dev/sda1 # Grow to partition size
e4defrag /mnt/data # Defragment
fstrim -v /mnt/data # Trim free space
```

# Appendix — Debugging Quick Reference

## debugfs & Tracing Commands

```
# debugfs
debugfs /dev/sda1 # Open
debugfs -R "stats" /dev/sda1 # Superblock
debugfs -R "ls -l /" /dev/sda1 # List root
debugfs -R "logdump -a" /dev/sda1 # Journal dump
debugfs -R "stat /path/to/file" /dev/sda1 # File status

# Tracing
echo 1 > /sys/kernel/debug/tracing/events/ext4/enable # Enable
cat /sys/kernel/debug/tracing/trace_pipe # Monitor
echo 0 > /sys/kernel/debug/tracing/events/ext4/enable # Disable

# Dynamic Debug
echo "file fs/ext4/* +p" > /sys/kernel/debug/dynamic_debug/control
echo "func ext4_map_blocks +p" > /sys/kernel/debug/dynamic_debug/control
```

# Appendix — Performance & BPF Quick Reference

```
# Performance
perf record -e ext4:* -a -g -- sleep 10 # Record
perf report # Analyse
perf stat -e ext4:* -a -- sleep 60 # Count events

# BPF
bpftrace -e 'kprobe:ext4_file_write { printf("%s\n", str(arg1->f_path.dentry->d_name.name)); }'
```

End of Presentation

# Linux Ext4 Internals

## Day 5: Debugging, Maintenance & Recovery — Complete


Instructor

Chandrashekar Babu

Email

training@chandrashekar.info

Website

chandrashekar.info | slashprog.com